

PIZZA HUT SALES ANALYSIS USING SQL

Exploring sales trends, revenue insights, and top-performing pizzas through SQL queries.

PROJECT CONTEXT

Goal

To dive into Pizza Hut's sales data to uncover actionable business insights using SQL.

Data Set

orders, orders_details,
pizzas, pizza_types



Focus Area

Revenue patterns,
customer preferences, and
order trends

*Let's explore the story the
data has to tell*

QUESTION 1

Code:

```
1 sql> • SELECT
2     COUNT(order_id) as total_orders
3 FROM orders;
```

Output:

total_orders	
21350	

OK, 1 record retrieved in 6.671ms

Retrieve the total number of orders placed.

For this one, I thought, “Okay, I need to know how many orders we got in total.” So I focused on the orders table because each order_id represents a unique order. Then I used the COUNT() function to count all the order IDs

Insight:

The total number of orders helps measure sales volume and customer activity. This metric is the foundation for further KPIs like revenue per order or average daily sales.

QUESTION 2

Code:

```
5  • SELECT
6      ROUND(SUM(orders_details.quantity * pizzas.price),2)
7      AS total_revenue
8  FROM
9      orders_details
10     JOIN
11     pizzas
12     ON pizzas.pizza_id = orders_details.pizza_id;
```

Output:

total_revenue

817860.05

OK, 1 record retrieved in 50.031ms

Calculate the total revenue generated from pizza sales

Here, I wanted to figure out how much money the pizza sales brought in. I knew that the orders_details table had quantities and the pizzas table had prices, so I joined them. Then I multiplied quantity by price for each order and summed everything up. I also rounded it to 2 decimal places so it looks neat.

Insight:

Total revenue reveals the business's overall performance. It acts as the key indicator for profitability and helps track financial growth.

QUESTION 3

Calculate the percentage contribution of each pizza type to total revenue.

Code:

```
16 SELECT
17     pizza_types.category,
18     (SUM(orders_details.quantity * pizzas.price) / (SELECT
19         ROUND(SUM(orders_details.quantity * pizzas.price),
20             2) AS total_revenue
21     FROM
22         orders_details
23     JOIN
24         pizzas ON pizzas.pizza_id = orders_details.pizza_id)) * 100 AS revenue
25 FROM
26     pizza_types
27     JOIN
28     pizzas ON pizza_types.pizza_type_id = pizzas.pizza_type_id
29     JOIN
30     orders_details ON orders_details.pizza_id = pizzas.pizza_id
31 GROUP BY pizza_types.category
32 ORDER BY revenue DESC;
```

Output:

category	revenue
Classic	26.90596025566967
Supreme	25.45631126009862
Chicken	23.955137556847287
Veggie	23.682590927384577
OK, 4 records retrieved in 139.217ms	

For this, I thought, “Which pizza categories are bringing in the most money?” So I joined pizza_types, pizzas, and orders_details to connect categories, prices, and quantities. I first calculated the total revenue using a subquery, then divided the revenue of each category by the total revenue and multiplied by 100 to get the percentage.

Insight:

This shows which pizza categories drive the most revenue. It helps prioritize production, inventory, and marketing toward high-performing segments.

QUESTION 4

Code:

```
35  • SELECT
36      pizza_types.name, pizzas.price
37  FROM
38      pizza_types
39      JOIN
40      pizzas
41      ON pizza_types.pizza_type_id = pizzas.pizza_type_id
42  ORDER BY pizzas.price DESC
43  LIMIT 1;
```

Output:

name	price
The Greek Pizza	35.95
OK, 1 record retrieved in 2.592ms	

Identify the highest-priced pizza

I wanted to see which pizza is the priciest. I joined pizza_types with pizzas so I could see the name with its price. Then I ordered the results by price descending and picked the top one using LIMIT 1.

Insight:

Identifying the premium pizza helps understand pricing strategy and how luxury offerings can affect the overall brand image or margin.

QUESTION 5

Code:

```
46  • SELECT
47      pizzas.size,
48      COUNT(orders_details.order_details_id) AS order_count
49  FROM
50      pizzas
51      JOIN
52      orders_details
53      ON pizzas.pizza_id = orders_details.pizza_id
54  GROUP BY pizzas.size
55  ORDER BY order_count DESC
56  LIMIT 1;
```

Output:

size	order_count
L	18526
OK, 1 record retrieved in 83.509ms	

Identify the most common pizza size ordered

Here, I asked myself, “Which size do people love the most?” I joined pizzas and orders_details to connect sizes with orders. Then I counted the number of orders for each size using COUNT() and grouped by size. Sorting in descending order and limiting to 1 gave me the most popular size.

Insight:

The most popular size helps optimize inventory planning, packaging, and pricing strategy...so typically “Large” pizzas dominate demand patterns.

QUESTION 6

List the top 5 most ordered pizza types

Code:

```
59  • SELECT
60      pizza_types.name, SUM(orders_details.quantity) AS quantity
61  FROM
62      pizza_types
63      JOIN
64      pizzas
65      ON pizza_types.pizza_type_id = pizzas.pizza_type_id
66      JOIN
67      orders_details
68      ON orders_details.pizza_id = pizzas.pizza_id
69  GROUP BY pizza_types.name
70  ORDER BY quantity DESC
71  LIMIT 5;
```

Output:

name	quantity	
The Classic Deluxe Pizza	2453	
The Barbecue Chicken Pizza	2432	
The Hawaiian Pizza	2422	
The Pepperoni Pizza	2418	
The Thai Chicken Pizza	2371	
OK, 5 records retrieved in 132.65ms		

I wanted to highlight the customer favorites. I joined pizza_types, pizzas, and orders_details to link pizza names with quantities. Summing up quantities and ordering descending, I then picked the top 5 using LIMIT.

Insight:

The top 5 pizzas represent customer favorites which is ideal for featured menu items, combo deals, or promotional focus.

QUESTION 7

Find total quantity of each pizza category ordered

Code:

```
76  • SELECT
77      pizza_types.name, SUM(orders_details.quantity) AS quantity
78  FROM
79      pizza_types
80      JOIN
81      pizzas
82      ON pizza_types.pizza_type_id = pizzas.pizza_type_id
83      JOIN
84      orders_details
85      ON orders_details.pizza_id = pizzas.pizza_id
86  GROUP BY pizza_types.name
87  ORDER BY quantity DESC;
```

Output:

name	quantity
The Classic Deluxe Pizza	2453
The Barbecue Chicken Pizza	2432
The Hawaiian Pizza	2422
The Pepperoni Pizza	2418
The Thai Chicken Pizza	2371
The California Chicken Pizza	2370
The Sicilian Pizza	1938
The Spicy Italian Pizza	1924
The Southwest Chicken Pizza	1917
The Big Meat Pizza	1914
The Four Cheese Pizza	1902
The Italian Supreme Pizza	1884

OK, 32 records retrieved in 125.063ms

For this, I thought, “Let’s see how many pizzas from each category were sold in total.” Using the same joins as before, I grouped by pizza category and summed the quantities to get the total per category.

Insight:

Understanding category-wise sales reveals which segments (Veg, Chicken, Supreme, etc.) are most demanded and essentially for product mix optimization.

QUESTION 8

Code:

```
90  • SELECT
91      HOUR(order_time) AS hour, COUNT(order_id) AS order_count
92  FROM
93      orders
94  GROUP BY HOUR(order_time);
95
```

Output:

hour	order_count	hour	order_count
14	1472	11	1231
15	1468	12	2520
16	1920	13	2455
17	2336		
18	2399		
19	2009		
20	1642		
21	1198		
22	663		
23	28		
10	8		
9	1		
OK, 15 records retrieved			

Determine the distribution of orders by hour

I wanted to know when most orders happen. I extracted the hour from order_time using HOUR() and then counted how many orders occurred in each hour using COUNT().

Insight:
Identifies peak business hours (typically dinner time) allowing targeted promotions or staffing optimization during high-demand periods.

QUESTION 9

Code:

```
97  • SELECT
98      category, COUNT(name)
99  FROM
100     pizza_types
101  GROUP BY category;
```

Output:

category	COUNT(name)
Chicken	6
Classic	8
Supreme	9
Veggie	9

OK, 4 records retrieved in 1.524ms

Category-wise distribution of pizzas

Here, I thought, “How many pizza options do we have in each category?” So I queried pizza_types and grouped by category to count the number of pizza names under each one.

Insight:

Helps understand product diversity and menu balance – how many options exist under each category like Classic, Veggie, or Chicken.

QUESTION 10

Code:

```
105 • SELECT
106     ROUND(AVG(quantity), 0)
107 FROM
108     (SELECT
109         orders.order_date, SUM(orders_details.quantity) AS quantity
110     FROM
111         orders
112     JOIN orders_details ON orders.order_id = orders_details.order_id
113     GROUP BY orders.order_date) AS order_quantity;
```

Output:

```
ROUND(AVG(quantity), 0)
138
OK, 1 record retrieved in 88.127ms
```

*Average number of pizzas ordered
per day*

I wanted to find daily sales trends. I first grouped orders by order_date and summed quantities per day in a subquery, then calculated the average of these daily totals using AVG() and rounded the result.

Insight:

Provides insight into average daily sales volume, very much crucial for forecasting demand, managing supply chain, and labor scheduling.

QUESTION 11

Code:

```
117 • SELECT
118     pizza_types.name,
119     SUM(orders_details.quantity * pizzas.price) AS revenue
120 FROM
121     pizza_types
122     JOIN
123     pizzas ON pizzas.pizza_type_id = pizza_types.pizza_type_id
124     JOIN
125     orders_details ON orders_details.pizza_id = pizzas.pizza_id
126 GROUP BY pizza_types.name
127 ORDER BY revenue DESC
128 LIMIT 3;
```

Output:

name	revenue
The Thai Chicken Pizza	43434.25
The Barbecue Chicken Pizza	42768
The California Chicken Pizza	41409.5
OK, 3 records retrieved in 102.019ms	

*Top 3 most ordered pizza types
based on revenue*

I aimed to find the pizzas making the most money. Joining all relevant tables, I multiplied quantity by price, summed by pizza name, and sorted descending. LIMIT 3 gave me the top three.

Insight:

Identifies the highest-grossing products which are also the key for sales focus and inventory prioritization.

QUESTION 12

Analyze the cumulative revenue generated over time.

Code:

```
131 •SELECT order_date,
132      SUM(revenue) OVER(order by order_date) as cum_revenue
133 FROM
134      (SELECT orders.order_date, SUM(orders_details.quantity * pizzas.price) AS revenue
135      FROM orders_details
136      JOIN pizzas
137      ON orders_details.pizza_id = pizzas.pizza_id
138      JOIN orders
139      ON orders.order_id = orders_details.order_id
140 GROUP BY orders.order_date) as sales;
```

Output:

order_date	cum_revenue
01/01/2015	2713.85000000000004
01/02/2015	5445.75
01/03/2015	8108.15
01/04/2015	9863.6
01/05/2015	11929.55
01/06/2015	14358.5
01/07/2015	16560.7
01/08/2015	19399.05
01/09/2015	21526.4
01/10/2015	23990.3500000000002
01/11/2015	25862.65
01/12/2015	27781.7

OK, 358 records retrieved in 95.202ms

I wanted to track how revenue grows over time. I first calculated daily revenue in a subquery, then used a window function SUM() OVER(ORDER BY order_date) to get cumulative revenue.

Insight:
Shows how revenue accumulates over time. It is useful for tracking growth patterns and monthly performance trends.

QUESTION 13

Code:

```
142 •select name, revenue from
143     (select category, name, revenue,
144      rank() over(partition by category order by revenue desc) as rn
145      from
146       (select pizza_types.category, pizza_types.name,
147        sum((orders_details.quantity)*pizzas.price) as revenue
148        from pizza_types join pizzas
149         on pizza_types.pizza_type_id = pizzas.pizza_type_id
150        join orders_details
151         on orders_details.pizza_id = pizzas.pizza_id
152        group by pizza_types.category, pizza_types.name) as a) as b
153     where rn <= 3
154     limit 3;
```

Output:

name	revenue
The Thai Chicken Pizza	43434.25
The Barbecue Chicken Pizza	42768
The California Chicken Pizza	41409.5
OK, 3 records retrieved in 125.065ms	

Top 3 pizzas by revenue per category

For this, I asked, “Which pizzas perform best in each category?” I joined pizza_types, pizzas, and orders_details, then used RANK() over PARTITION BY category to rank them by revenue within each category, and filtered the top 3.

Insight:

Highlights star performers within each category. Useful for category-level menu optimization and promotional campaigns.

CONCLUSION

The top half of the slide features a dark blue background with the word 'CONCLUSION' in large, bold, light yellow letters. Scattered across this background are several small, white, hand-drawn style icons of pizza slices, some with toppings like pepperoni and mushrooms.

This project demonstrates how SQL can turn raw data into actionable business insights. By analyzing Pizza Hut's sales data, I uncovered patterns in customer preferences, revenue drivers, and order trends.

The exercise strengthened my skills in joins, aggregations, subqueries, and window functions, while also highlighting the importance of data-driven decision-making in a real-world business context.

KEY TAKE AWAYS & RESOURCES

SQL can transform raw data into actionable business insights.

- Joins, aggregations, subqueries, and window functions are powerful tools for analyzing real-world datasets.
- Understanding revenue patterns, top-selling products, and customer trends helps make data-driven business decisions.
- Data exploration is just as important as the queries – always ask: “What story is this data telling?”

- **GITHUB REPOSITORY**

- **KAGGLE DATASET**

- **PORTFOLIO**

TOOLS USED

- MySQL
- Kaggle
- VSCode
- GitHub